

Федеральное государственное автономное образовательное учреждение высшего
Образования «Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники

Лабораторная работа №5
По программированию
Вариант 864

Работу выполнил:

Лыба Данил Вячеславович

Группа Р3110

Преподаватель, принимающий работу:

Выдра Андрей Михайлович

г. Санкт-Петербург 2026 г.

Содержание

Текст задания	3
Диаграмма классов	5
Исходный код программы	5
Вывод.....	5

Текст задания

Реализовать консольное приложение, которое реализует управление коллекцией объектов в интерактивном режиме. В коллекции необходимо хранить объекты класса `Flat`, описание которого приведено ниже.

Разработанная программа должна удовлетворять следующим требованиям:

- Класс, коллекцией экземпляров которого управляет программа, должен реализовывать сортировку по умолчанию.
- Все требования к полям класса (указанные в виде комментариев) должны быть выполнены.
- Для хранения необходимо использовать коллекцию типа `java.util.Vector`
- При запуске приложения коллекция должна автоматически заполняться значениями из файла.
- Имя файла должно передаваться программе с помощью: **переменная окружения**.
- Данные должны храниться в файле в формате `xml`
- Чтение данных из файла необходимо реализовать с помощью класса `java.util.Scanner`
- Запись данных в файл необходимо реализовать с помощью класса `java.io.FileWriter`
- Все классы в программе должны быть задокументированы в формате `javadoc`.
- Программа должна корректно работать с неправильными данными (ошибки пользовательского ввода, отсутствие прав доступа к файлу и т.п.).

В интерактивном режиме программа должна поддерживать выполнение следующих команд:

- `help`: вывести справку по доступным командам
- `info`: вывести в стандартный поток вывода информацию о коллекции (тип, дата инициализации, количество элементов и т.д.)
- `show`: вывести в стандартный поток вывода все элементы коллекции в строковом представлении
- `add {element}`: добавить новый элемент в коллекцию
- `update id {element}`: обновить значение элемента коллекции, id которого равен заданному
- `remove_by_id id`: удалить элемент из коллекции по его id
- `clear`: очистить коллекцию
- `save`: сохранить коллекцию в файл
- `execute_script file_name`: считать и исполнить скрипт из указанного файла. В скрипте содержатся команды в таком же виде, в котором их вводит пользователь в интерактивном режиме.
- `exit`: завершить программу (без сохранения в файл)
- `remove_last`: удалить последний элемент из коллекции
- `remove_lower {element}`: удалить из коллекции все элементы, меньшие, чем заданный
- `reorder`: отсортировать коллекцию в порядке, обратном нынешнему
- `average_of_number_of_rooms`: вывести среднее значение поля `numberOfRooms` для всех элементов коллекции
- `count_greater_than_house house`: вывести количество элементов, значение поля `house` которых больше заданного
- `filter_by_new new`: вывести элементы, значение поля `new` которых равно заданному

Формат ввода команд:

- Все аргументы команды, являющиеся стандартными типами данных (примитивные типы, классы-оболочки, String, классы для хранения дат), должны вводиться в той же строке, что и имя команды.
- Все составные типы данных (объекты классов, хранящиеся в коллекции) должны вводиться по одному полю в строку.
- При вводе составных типов данных пользователю должно показываться приглашение к вводу, содержащее имя поля (например, "Введите дату рождения:")
- Если поле является enum'ом, то вводится имя одной из его констант (при этом список констант должен быть предварительно выведен).
- При некорректном пользовательском вводе (введена строка, не являющаяся именем константы в enum'e; введена строка вместо числа; введенное число не входит в указанные границы и т.п.) должно быть показано сообщение об ошибке и предложено повторить ввод поля.
- Для ввода значений null использовать пустую строку.
- Поля с комментарием "Значение этого поля должно генерироваться автоматически" не должны вводиться пользователем вручную при добавлении.

Описание хранимых в коллекции классов:

```
public class Flat {
    private Long id; //Поле не может быть null, Значение поля должно быть больше 0, Значение этого поля должно быть уникальным, Значение этого поля должно генерироваться авт
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.time.LocalDate createDate; //Поле не может быть null, Значение этого поля должно генерироваться автоматически
    private double area; //Значение поля должно быть больше 0
    private int numberOfRooms; //Значение поля должно быть больше 0
    private Boolean new; //Поле может быть null
    private Furnish furnish; //Поле может быть null
    private Transport transport; //Поле может быть null
    private House house; //Поле может быть null
}
public class Coordinates {
    private float x;
    private long y;
}
public class House {
    private String name; //Поле не может быть null
    private long year; //Значение поля должно быть больше 0
    private long numberOfFlatsOnFloor; //Значение поля должно быть больше 0
    private Integer numberOfLifts; //Значение поля должно быть больше 0
}
public enum Furnish {
    DESIGNER,
    BAD,
    LITTLE;
}
public enum Transport {
    NONE,
    NORMAL,
    ENOUGH;
}
}
```



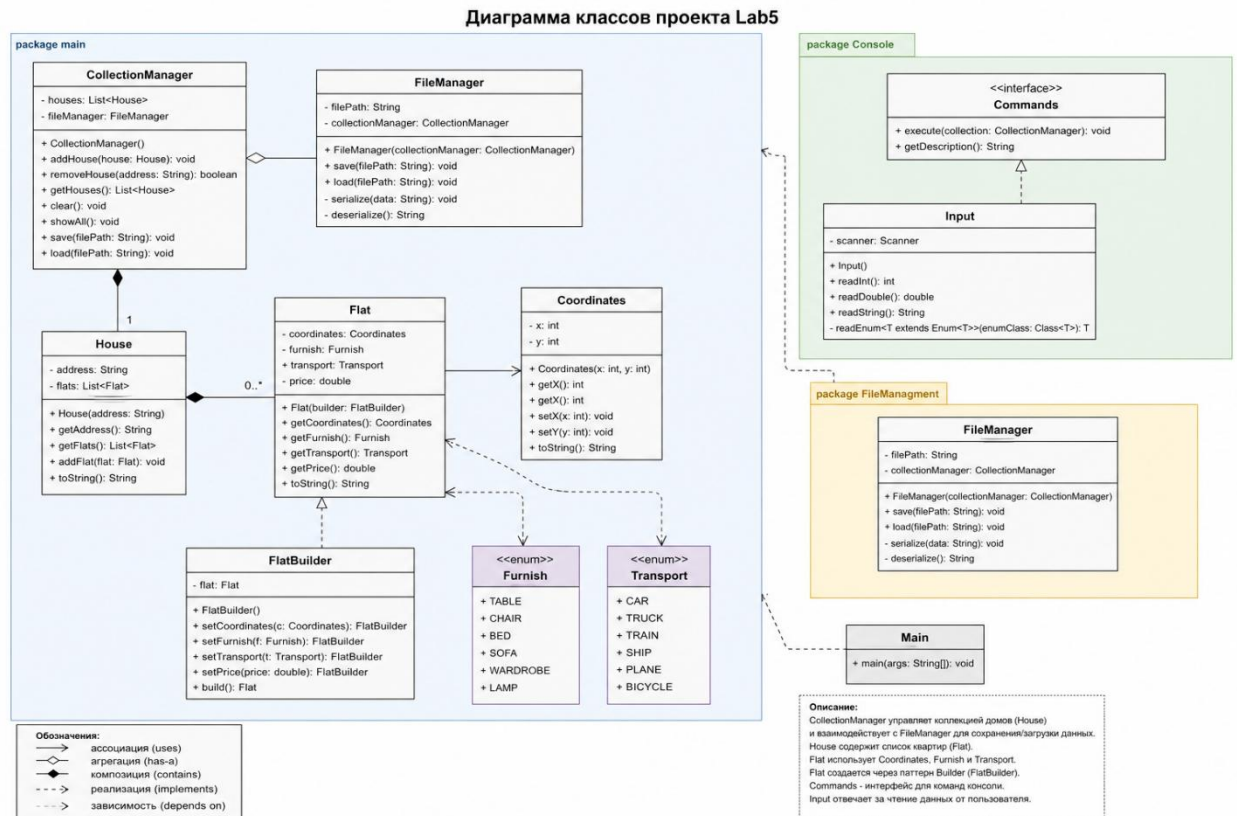
Отчёт по работе должен содержать:

1. Текст задания.
2. Диаграмма классов разработанной программы.
3. Исходный код программы.
4. Выводы по работе.

Вопросы к защите лабораторной работы:

1. Коллекции. Сортировка элементов коллекции. Интерфейсы `java.util.Comparable` и `java.util.Comparator`.
2. Категории коллекций - списки, множества, очереди, отображения (словари). Интерфейсы `java.util.List`, `java.util.Set`, `java.util.Queue`, `java.util.Map` и их реализации.
3. Параметризованные типы. Создание параметризуемых классов. Wildcard-параметры.
4. Классы-оболочки. Назначение, область применения, преимущества и недостатки. Автоупаковка и автораспаковка.
5. Потоки ввода-вывода в Java. Байтовые и символьные потоки. "Цепочки" потоков (Stream Chains).
6. Работа с файлами в Java. Класс `java.io.File`.
7. Пакет `java.nio` - назначение, основные классы и интерфейсы.
8. Утилита `javadoc`. Особенности автоматического документирования кода в Java.

Диаграмма классов



Исходный код программы

<https://github.com/DanexaPlay/Lab5>

Вывод

При выполнении лабораторной работы я создал собственное консольное приложение для управления коллекцией, научился работать с файлами и паттернами проектирования.